

Arquitetura do Jogo - Grandmas Garden

Contents

Arquitetura do Jogo – Grandma’s Garden	1
Referência Arquitetural Completa (Alto Nível)	1
0. Introdução e Filosofia	1
1. Visão Geral	3
2. Decisões Arquiteturais Fundamentais	7
3. C4 Diagrams	13
4. Modelos de Dados	18
5. Diagramas de Classe	21
6. Diagramas de Sequência	22
7. Máquinas de Estado	28
8. Wireframes	29
9. Fluxo de Dados	30
10. Especificação de APIs	33
11. Análise de Decisões de Implementação	36
12. Fluxo de Gameplay Completo	39
13. Formato de Serialização	41
14. Resumo de Constraints	44

Arquitetura do Jogo – Grandma’s Garden

Referência Arquitetural Completa (Alto Nível)

0. Introdução e Filosofia

0 que é o Grandma’s Garden

Grandma’s Garden é um jogo de puzzle físico 2D onde jogadores criam e resolvem fases usando um conjunto diversificado de peças mecânicas – bolas, engrenagens, motores, correias, cordas, lasers, emissores, consumidores elétricos e muito mais. O jogo combina a criatividade de um editor de fases com a satisfação de ver máquinas físicas funcionando de forma determinística.

A essência do jogo está em dois fluxos complementares: **criar** fases no editor (usando ferramentas visuais com gizmos, handles e diálogos) e **resolver** puzzles no puzzle editor, onde o jogador recebe um inventário limitado de peças e deve encontrar a configuração correta para resolver o desafio.

A Dualidade Criativa

O jogo bifurca-se em duas experiências distintas:

Fase de Criação (Level Editor): O creator coloca peças na tela, conecta-as com ropes e belts, define condições de solução, configura state machines e testando inline. É um ambiente OOP completo com undo/redo, validação, e preview visual. O foco é liberdade criativa e construção de puzzles.

Fase de Resolução (Puzzle Editor): O player recebe uma fase com um inventário limitado de peças (subconjunto das que o creator colocou). O objetivo é posicionar essas peças de forma a satisfazer as condições de solução. O foco é raciocínio lógico e experimentação.

Ambas as experiências compartilham o mesmo motor de simulação, garantindo que o que funciona no editor funcionará no puzzle editor e vice-versa.

Filosofia de Design

Determinismo Acima de Tudo

Cada simulação deve ser **reproduzível**. Dado o mesmo level_data e o mesmo seed do RNG, a simulação produz exatamente os mesmos resultados em qualquer execução, em qualquer máquina. Isso permite: - **Debugging preciso:** um bug repropduz-se sempre, em qualquer lugar - **Testes automatizados:** deterministic tests com 50+ runs para validar consistência - **Gameplay previsível:** o jogador pode confiar que suas soluções são estáveis

Dados Sobre Código

O comportamento das fases é governado por **chain scripts** – estruturas de dados JSON que definem condições e ações. Não há código hardcoded para lógicas de puzzle. Isso significa que: - Designers podem criar puzzles sem escrever código - O motor de simulação é genérico e extensível - Novas lógicas podem ser adicionadas sem modificar código C#/GDScript

Isolamento de Responsabilidades

Cada subsistema tem uma responsabilidade clara e limites bem definidos: - PartData existe apenas no editor (OOP, com callbacks, gizmos, handles) - Part existe apenas na simulação (data-driven, stateless, deterministic) - ChainManager avalia regras sem conhecer detalhes de física - PhysicsEngine executa física sem saber de game logic

Physics como Sistema de Primeira Classe

A física não é um detalhe de implementação – é o **núcleo do gameplay**. O motor de física é deterministic, opera em fixed-point (Q8.8), e executa 13 passes por tick em ordem estrita. Cada peça pode ter state machines, triggers, e comportamentos que interagem com o sistema físico.

1. Visão Geral

1.1 Arquitetura Bifurcada – Três Contextos Mutuamente Exclusivos

O jogo opera em **três contextos funcionalmente separados**, cada um com seu próprio conjunto de objetos, responsabilidades e ciclo de vida. Importante: **nenhum contexto interage diretamente com outro durante execução** – a comunicação ocorre apenas via serialização (Editor → Simulação) ou sinais (Simulação → UI).

Contexto	Paradigma	Objetos	Propósito
Level Editor	OOP App	EditorController, WorldState, PartData[], PhysicsSolver, UndoSystem	Criar e editar fases – gizmos, handles, diálogos, undo/redo, preview inline
Puzzle Editor	OOP App	EditorController, WorldState, PartData[] (inventário), SolutionChecker	Resolver puzzles – posiciona peças do inventário, verifica solução
Simulação	Data-Driven	SimulationRunner, Board, Part[], PhysicsEngine, ChainManager	Playback determinístico – isolado em SubViewport, não interativo

Por que três contextos e não dois?

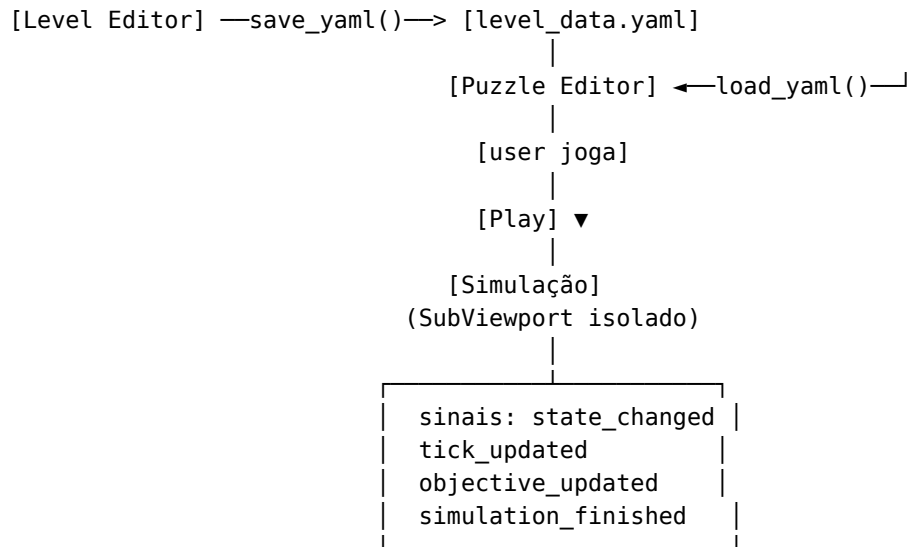
A bifurcação entre Level Editor e Puzzle Editor é **fundamental**, não um detalhe de implementação:

Level Editor é um ambiente de **autoria**. O creator precisa de acesso total a todas as peças da fase, ferramentas de conexão, diálogos de state machine, condições de solução, e preview inline. É um aplicativo OOP completo com sua própria máquina de estados de ferramentas (SELECT, PLACE, DELETE, CONNECT, PREVIEW, LEVEL_INFO).

Puzzle Editor é um ambiente de **resolução**. O player recebe apenas as peças do inventário (subconjunto das que o creator colocou). Não pode adicionar peças além do inventário, não pode editar state machines, não pode alterar conexões. O foco é resolver o puzzle, não criar.

Simulação é o motor que **executa** a física. Roda isolado em um SubViewport, não tem acesso a nenhum objeto do editor. Apenas recebe level_data (serializado), executa o loop determinístico, e emite sinais de volta para a UI.

Fluxo de Dados Entre Contextos



A serialização é o **único ponto de contato** entre contextos. Não há estado compartilhado em runtime. O que não é serializado (callbacks, gizmos, `undo_stack`, `part_nodes`) é **editor-only** e não existe na simulação.

1.2 Princípios Fundamentais

Quatro princípios orientam todas as decisões arquiteturais do projeto:

Princípio 1: Dual Part Representation

Cada peça física no jogo existe em **duas representações distintas**:

- **PartData** (editor): Objeto OOP com 282 campos, callbacks, gizmos, handles, undo/redo. Existe apenas no contexto do editor. Tem referências a callbacks (`on_collision`, `on_state_change`), posições de handles de resize, e integração com o sistema de undo.
- **Part** (simulação): Estrutura data-driven com ~60 campos, sem callbacks, sem estado Godot. Existe apenas no contexto de simulação. Contém apenas dados necessários para física determinística.

Por que não uma única classe?

Porque os requisitos são fundamentalmente conflitantes: - O editor precisa de **callbacks** (Quando o usuário arrasta uma peça, os sprites precisam atualizar? Quando o `state_counter` muda, o gizmo precisa revalidar?). `PartData` tem `on_collision` e `on_state_change` como Callable.

- A simulação precisa de **determinismo** (Não pode haver side effects além dos dados. O mesmo `level_data` deve produzir o mesmo resultado). Part não tem callbacks – apenas dados.

Se usássemos uma única classe, teríamos que escolher entre callbacks (que quebram determinismo) ou sem callbacks (que quebram o editor).

Princípio 2: Loop de Simulação ≠ Godot Loop

O `_process()` do Godot **não é** o loop de simulação. É apenas um **pacemaker externo** que mede wall-clock time.

O problema: O Godot `_process()` varia com frame rate. Em uma máquina rápida pode rodar 144fps, em uma lenta 30fps. Se a física dependesse de `_process()`, o comportamento seria diferente em cada máquina.

A solução:

```
_process(delta):
    tick_accum_ms += delta * 1000
    while tick_accum_ms >= 16.667: # 1000/60
        tick_accum_ms -= 16.667
        _internal_tick()
```

A física sempre avança em **incrementos fixos de 16.667ms** (60Hz), independente do frame rate. Se a máquina é rápida, o `while` roda várias vezes por frame. Se é lenta, o `while` pode rodar zero vezes (e o jogo “congela” visualmente, mas a física não acelera).

Princípio 3: Fluxo de Dados Unidirecional

O fluxo de dados é **estritamente unidirecional**:

Editor —serialização—> Simulação —sinais—> UI
(`level_data`)

O que flui do Editor para Simulação: - Setup data: posições iniciais, state counters, conexões, flags - Chain script: condições e ações (JSON array) - Solution conditions: zona de vitória, estado requerido

O que flui da Simulação para Editor: - Sinais: `state_changed(state)`, `tick_updated(tick, max)`, `objective_updated(primary, secondary)`, `simulation_finished(success, tick, events)` - **Nada de estado runtime** (posições, velocidades) volta para o editor

Este isolamento garante que a simulação não pode “poluir” o estado do editor. Bugs na simulação não afetam o editor.

Princípio 4: Determinismo Completo

Mesma entrada → mesmo resultado, em qualquer execução, em qualquer máquina.

O determinismo é garantido por: 1. **Fixed-point arithmetic** (Q8.8): Sem floating point, sem arredondamentos variáveis 2. **Fixed tick rate** (60Hz): Sempre o mesmo número de ticks por segundo 3. **LCG RNG com seed fixa**: O RNG é determinístico – mesma seed = mesma sequência de números 4. **Ordem estrita de passes**: physics.step() executa 13 passes em ordem fixa 5. **Sem callbacks no loop de física**: Part.advance_state() não chama callbacks externos

2. Decisões Arquiteturais Fundamentais

Esta seção documenta as **5 decisões de design mais críticas** do projeto – decisões que afetam profundamente a estrutura do código e que não podem ser revertidas facilmente. Cada decisão inclui o **contexto** (por que foi necessária), a **solução adotada**, e os **trade-offs** envolvidos.

Decisão 1: Dual Part Representation (PartData vs Part)

Contexto: O editor precisa de callbacks, gizmos, handles, undo/redo e estado Godot. A simulação precisa de determinismo, sem side effects.

Problema Original: Uma única classe Part que tentasse servir ambos os propósitos teria que: - Suportar callbacks (on_collision, on_state_change) para o editor - Mas callbacks introduzem side effects que quebram determinismo - Sem callbacks, o editor não consegue atualizar gizmos quando state_counter muda

Solução Adotada: Duas classes completamente separadas: - PartData (editor): 282 campos, callbacks Callable, gizmos, handles, undo system. Referências a nós Sprite2D. - Part (simulação): ~60 campos, nenhum callback, estrutura flat para serialização. Apenas setup data + physics state.

O que é comum (superfície de interseção):

```
part_type, x, y, width_1, height_1, flags_1, connected_1/2,  
state_counter, sub_counter, state_limit_*, current_state,  
mass, cor_q8, friction_q8, collision_radius, facing
```

O que é exclusivo de PartData:

name, appearance, behavior, rotation_angle, angular_velocity,
motor_connected, density, elasticity, friction_extra,
gravity_category, max_gravity, wind_force, gravity_buoyancy,
outlet_plugged_1/2, belt_connected_part_1/2,
rope_1/2_anchor_x/y, pulley_rope_1/2_connect_x/y, rope_index,
explosion_timer, explosion_radius, is_moving, is_solved,
on_collision, on_state_change (Callable)

O que é exclusivo de Part:

pos_x, pos_y, vel_x, vel_y, is_powered, destroyed, spawns,
rope_pull_x, rope_pull_y, triggers, destructible,
archetype (enum), _rng_state, _countdown

Trade-offs: - **Duplicação de dados:** Cada part na memória tem duas cópias (PartData no editor, Part na simulação) - **Complexidade de serialização:** Bridge entre os dois formatos requer mapeamento cuidadoso - **Benefício:** Isolamento completo – bugs em um contexto não afetam o outro

Decisão 2: Loop de Simulação Separo do Godot Loop

Contexto: O `_process()` do Godot varia com frame rate (30-144fps tipicamente). A física precisa de 60Hz fixo determinístico.

Problema Original: Se a física dependesse diretamente de `_process(delta)`: - Máquina rápida (144fps): física roda 144 vezes por segundo – demasiado rápido - Máquina lenta (30fps): física roda 30 vezes por segundo – demasiado lento - Comportamento diferente em cada máquina, quebrando determinismo

Solução Adotada: `_process()` é apenas um **pacemaker wall-clock** que acumula tempo e dispara ticks internos:

```
var tick_accum_ms: float = 0.0
```

```
func _process(delta):  
    tick_accum_ms += delta * 1000 # delta em segundos, converte para ms  
    while tick_accum_ms >= 16.667: # 1000ms / 60 = 16.667ms  
        tick_accum_ms -= 16.667  
        _internal_tick() # physics.step() + chain.evaluate_chain()
```

Por que while e não if: - Se a máquina é rápida, pode acumular tempo suficiente para rodar múltiplos ticks em um frame - Se a máquina é lenta ($\text{delta} > 16.667\text{ms}$), vai rodar zero ticks (e o jogo congela visualmente, mas a física não acelera além do normal)

Trade-offs: - **Visual decoupling:** O que aparece na tela pode ser mais lento que a física real (se a renderização não acompanhar) -

Benefício: Determinismo garantido, física sempre em 60Hz independente de frame rate

Decisão 3: Chain Script como Motor de Gameplay

Contexto: O comportamento das fases é definido por designers, não por programadores. Regras de puzzle precisam ser flexíveis e editáveis sem código.

Problema Original: Se a lógica de puzzle fosse hardcoded em GDScript: - Cada novo tipo de puzzle requereria mudança de código - Designers precisariam pedir a programadores para modificar comportamento - Testar diferentes lógicas seria lento e arriscado

Solução Adotada: Chain script – um array de regras JSON avaliadas por ChainManager:

```
[
  {
    "when": {"type": "overlap", "a": "ball", "b": "laser"},
    "actions": [{"type": "win", "msg": "Level Complete!"}],
    "mode": "once"
  },
  {
    "when": {"type": "prop_gt_ref", "part": "ball", "prop": "state_counter",
    "ref_part": "motor", "ref_prop": "state_counter", "offset": 2},
    "actions": [{"type": "set_flag", "name": "ball_spinning", "value": true}],
    "mode": "repeat"
  }
]
```

7 tipos de condições: - tick: Sempre verdadeiro (executa todo tick) - flag: Verifica valor de uma flag - and: Todas as sub-condições precisam ser verdadeiras - prop_ge: Property >= valor - prop_eq: Property == valor - prop_gt_ref: Property > reference_property + offset - overlap: AABB overlap entre duas partes - solution: board.solution.check()

6 tipos de ações: - set_flag: Define flag _flags[name] = value - inc: Incrementa property p[prop] += value - set: Define property p[prop] = value - delta: Idem a inc, mas com lógica especial para posição x - log: Adiciona ao event_log e emite signal - win: Emite chain_completed (fim de jogo)

Modos: - once: Executa apenas uma vez (marca em _fired[]) - repeat: Executa todo tick em que a condição é verdadeira

Importante: Nenhum part reference hardcoded. Todas as referências a parts são por nome ("part": "ball"), resolvidas em runtime via `_find_part(part_name)`.

Trade-offs: - **Flexibilidade:** Designers criam puzzles sem código - **Debugging mais difícil:** Regras são dados, não código – erros são mais obscuros - **Performance:** Avaliar regras todo tick tem custo, mas é $O(n)$ onde n = número de regras

Decisão 4: State Machines como Sistema de Primeira Classe

Contexto: Quase todas as peças do jogo têm comportamento stateful – balls mudam de cor com base em `state_counter`, gears rotacionam, motores ligam/desligam, emissores mudam comportamento.

Problema Original: Se cada part type implementasse sua própria lógica de estado: - Muita duplicação de código - Comportamentos inconsistentes entre tipos diferentes - Impossível fazer deterministic RNG para delays

Solução Adotada: Todo part tem uma **state machine padronizada** implementada em `Part.advance_state()`:

```
state_counter == 0:
    - RNG delay: _countdown = random [40, 239] ticks
    - _rng_state & 1 == 0 → deterministic path (state_counter += 1)
    - _rng_state & 1 == 1 → random path (aguarda countdown)

state_counter != 0:
    - deterministic increment a cada tick
    - wrap at state_limit_upper → state_limit_reset

sub_counter > 0:
    - countdown mode: sub_counter-- por tick
    - quando chega a 0, volta para state_counter increment

on_collision() callback:
    - Se triggers.has("collision"): sempre incrementa state_counter
    - Override do RNG delay --- colisão sempre avança estado
```

Deterministic LCG RNG:

```
_rng_state = (_rng_state * 0x343FD + 0x269EC3) & 0xFFFFFFFF
_randint(lo, hi) = lo + (_rng_state % (hi - lo + 1))
```

Por que não Godot's RandomNumberGenerator? - Godot RNG não é determinístico por padrão - Precisamos de reprodutibilidade completa

entre execuções - LCG com seed fixa é a solução padrão para deterministic gaming

Trade-offs: - **Complexidade:** A state machine tem muitos edge cases (wrap, clamp, sub_counter) - **Benefício:** Comportamento consistente e previsível em todos os 278 tipos de peça

Decisão 5: Unidirectional Data Flow com Bridge de Serialização

Contexto: Editor e simulação são contextos completamente separados. Precisa haver comunicação, mas sem acoplamento direto.

Problema Original: Se editor e simulação compartilhassem objetos em runtime: - Simulação poderia modificar estado do editor acidentalmente - Bugs na física poderiam corromper estruturas do editor - Impossível fazer reset limpo da simulação sem afetar o editor

Solução Adotada: Bridge de serialização unidirecional:

Editor —_snapshot_world()—> level_data Dictionary
(setup data only)

level_data —SimulationRunner.init()—> Part[] (novo, isolado)
chain_script (copiado)
SolutionCondition (copiado)

Simulação —sinais—> UI (play/pause/stop, tick, finished)
(sem acesso a objetos do editor)

O que é serializado (setup data):

part_type, x, y, width_1/2, height_1/2,
flags_1, connected_1/2,
state_counter, state_limit_*, current_state,
mass, cor_q8, friction_q8, collision_radius, facing

O que NÃO é serializado (editor-only):

undo_stack, redo_stack (50 níveis de undo)
gizmo positions, handle_drag_idx, drag_offset
world.on_collision, on_state_change (Callable)
part_nodes (Sprite2D refs)
dialog_active, is_dirty, level_title
AnmRenderer instance, rect_tool
solver, checker instances

Sinais da simulação (feedback only):

signal state_changed(state: int) # RUNNING, PAUSED, SUCCESS, FAIL
signal tick_updated(tick: int, max: int) # UI updates tick counter

```
signal objective_updated(primary: bool, secondary: bool) # objective checkboxes  
signal simulation_finished(success: bool, tick_count: int, events: Array)
```

Trade-offs: - **Performance:** Serialização tem custo (mas é only-on-play, não todo frame) - **Isolamento:** Bugs não atravessam a bridge
- **Complexidade:** Dois formatos de dados (PartData vs Part) requerem mapeamento

3. C4 Diagrams

2.1 C4 Context – Sistema Completo

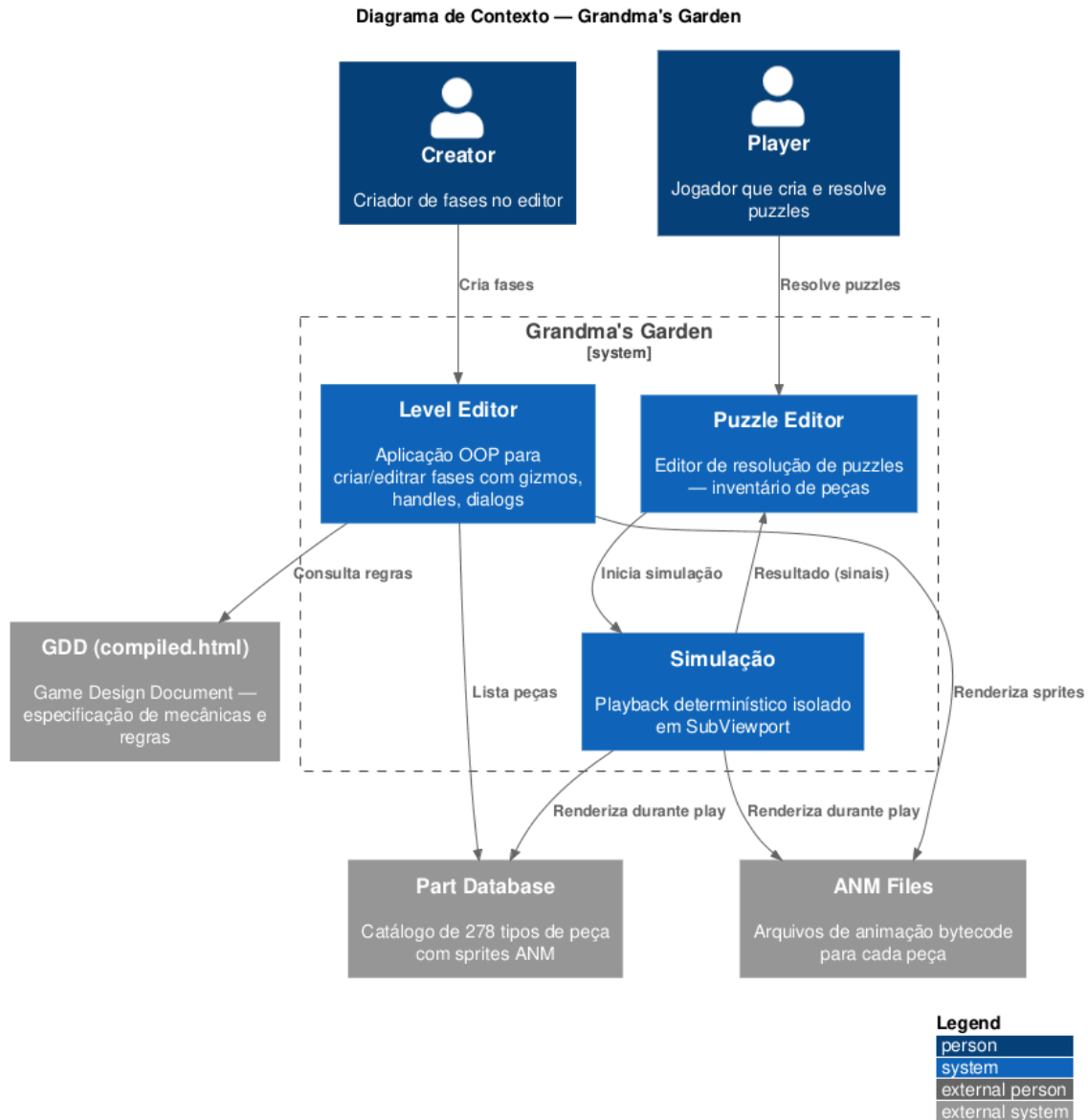


Figure 1: C4_Context

2.2 C4 Container – Arquitetura de Containers

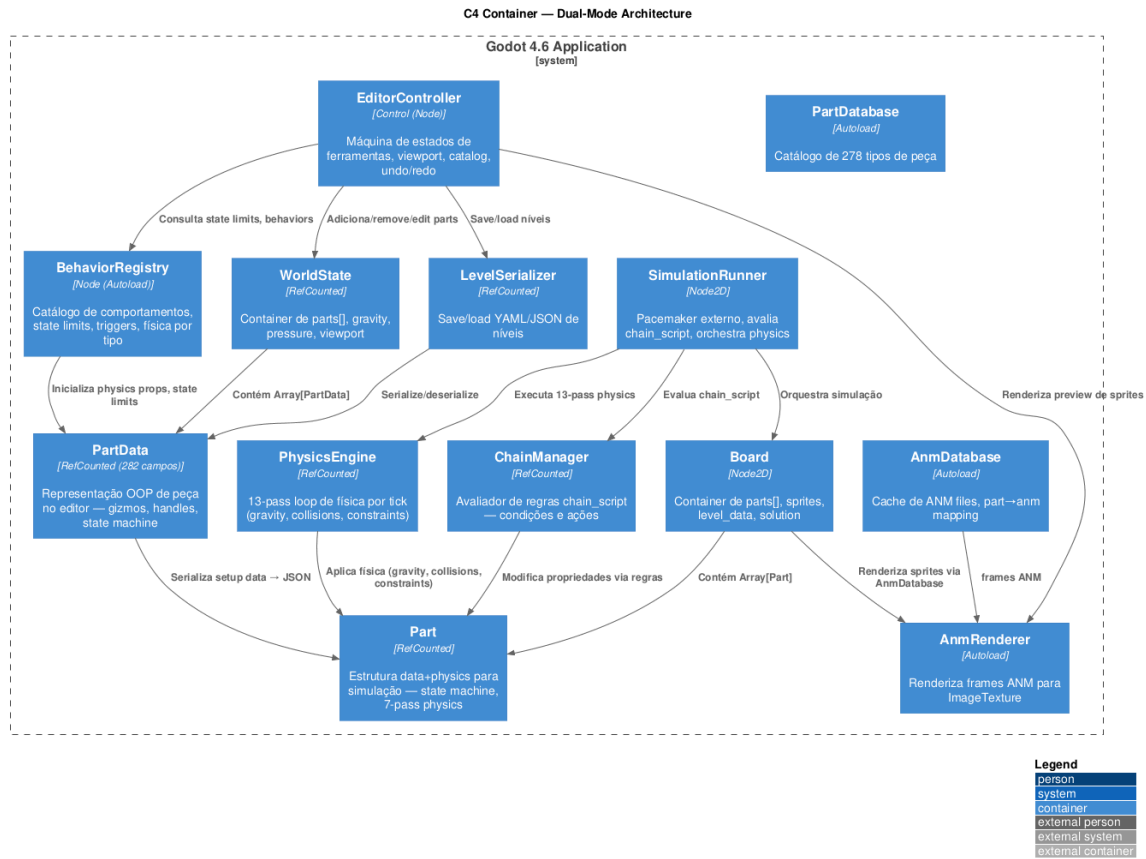
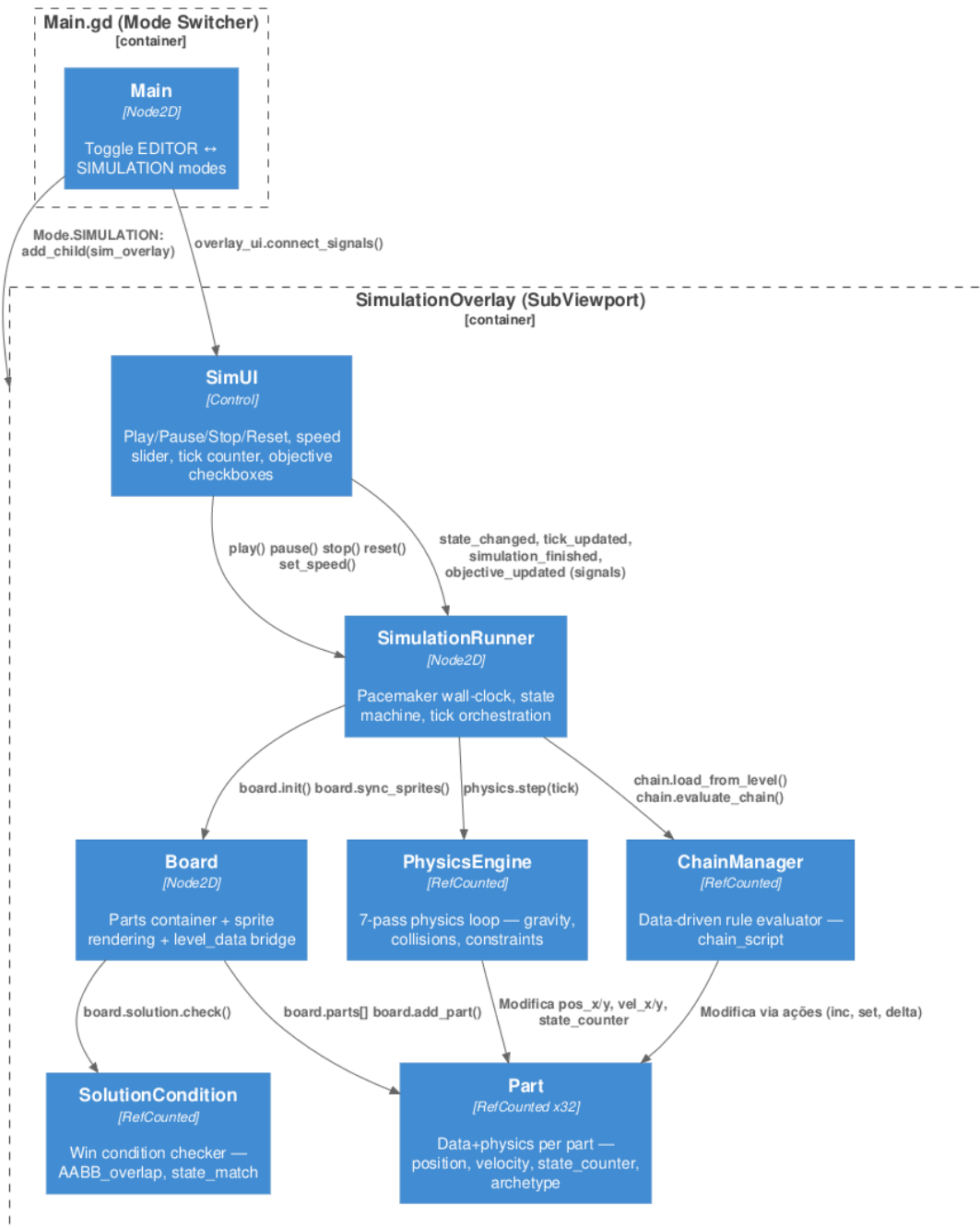


Figure 2: C4_Container

2.3 C4 Component – Simulação (Isolated SubViewport)

C4 Component — Simulation Isolated in SubViewport



Legend
person
system
container
component
external person
external system
external container
external component

Figure 3: C4_Simulation_Component

2.4 C4 Component – Editor

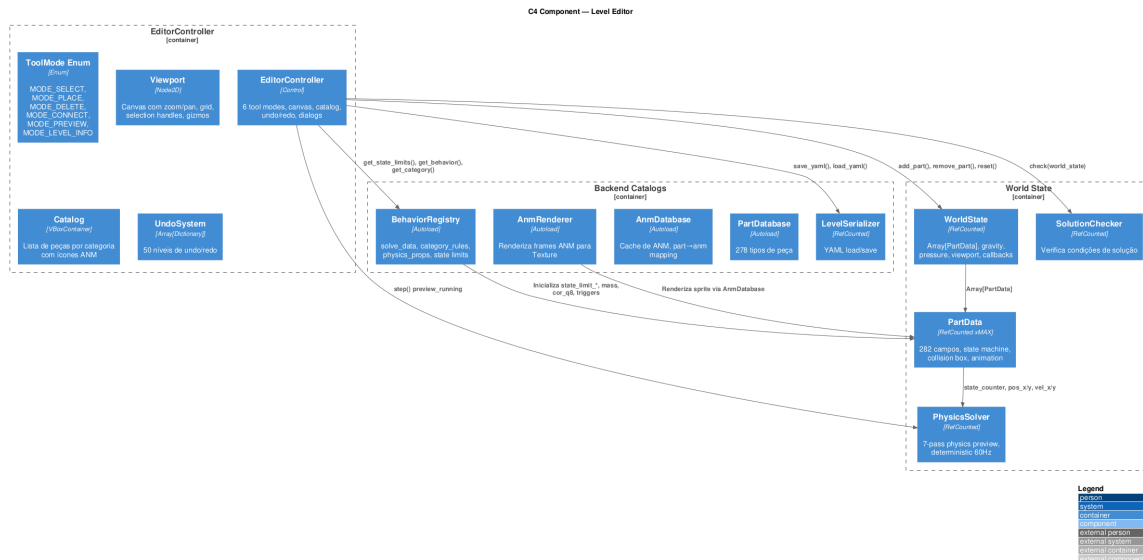


Figure 4: C4_Editor_Component

4. Modelos de Dados

3.1 Simulação – Part (Data-Driven, RefCounted)

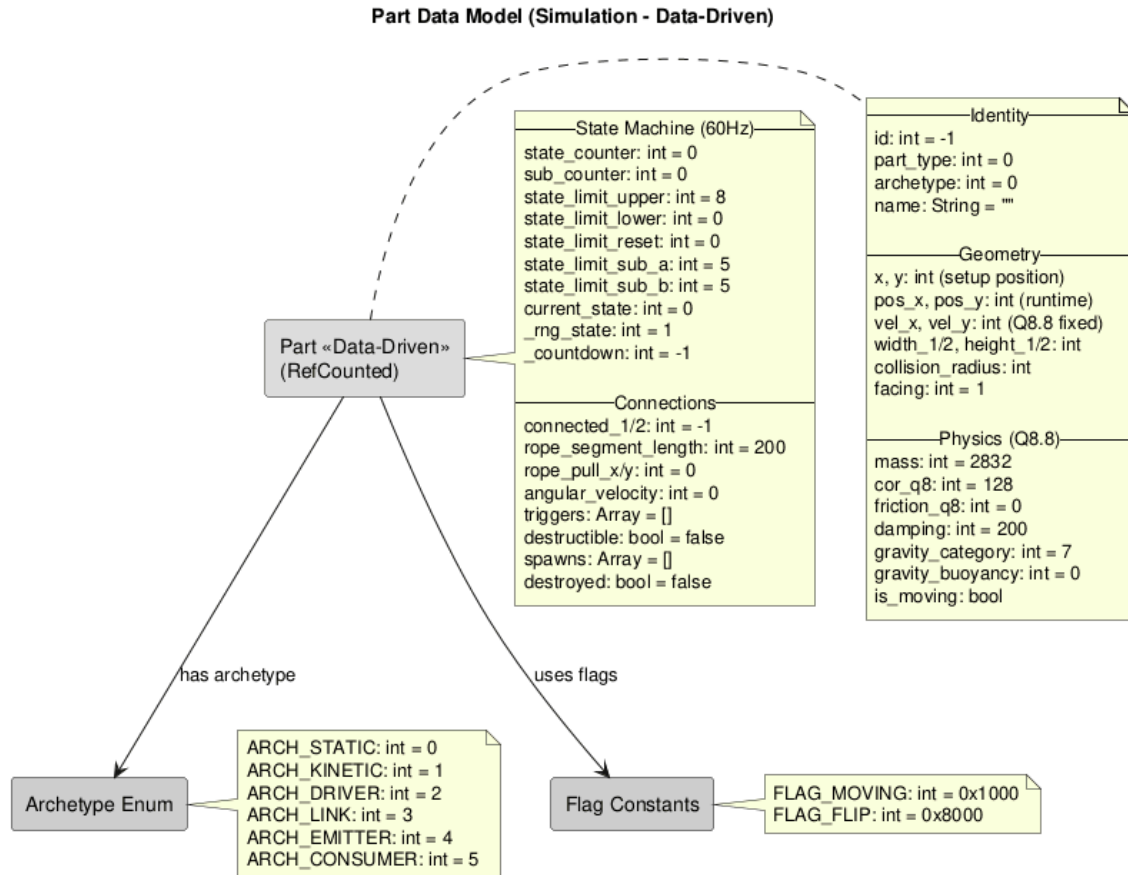


Figure 5: Part_Data_Model

3.2 Editor – PartData (OOP, RefCounted)

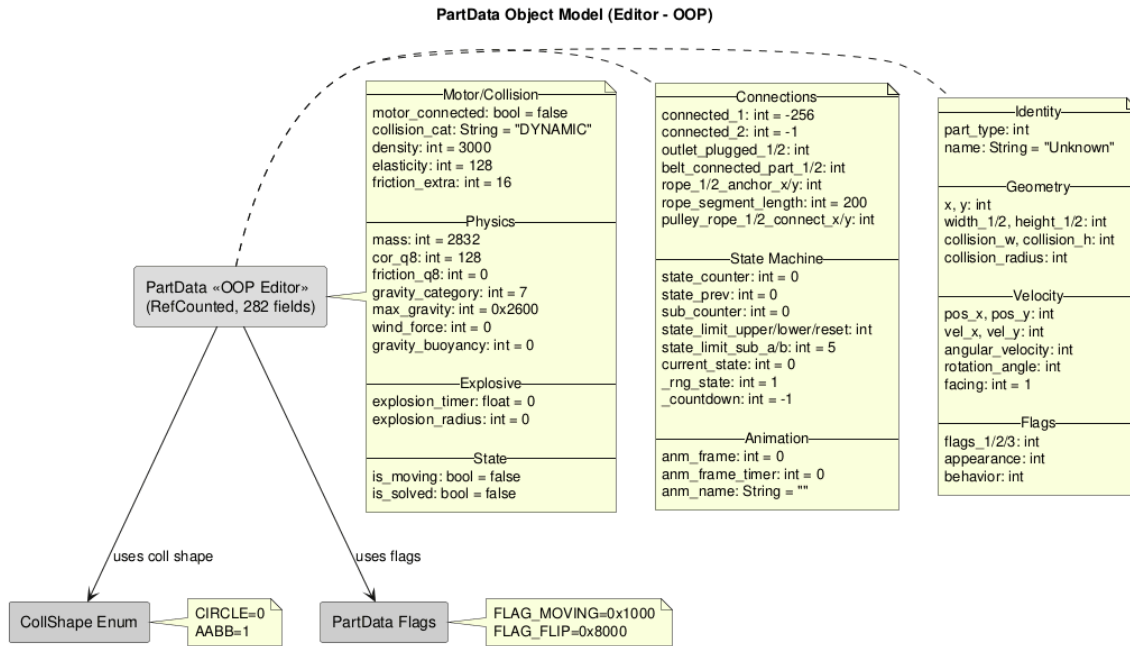


Figure 6: PartData_Object_Model

3.3 Chain Script – Data-Driven Rules

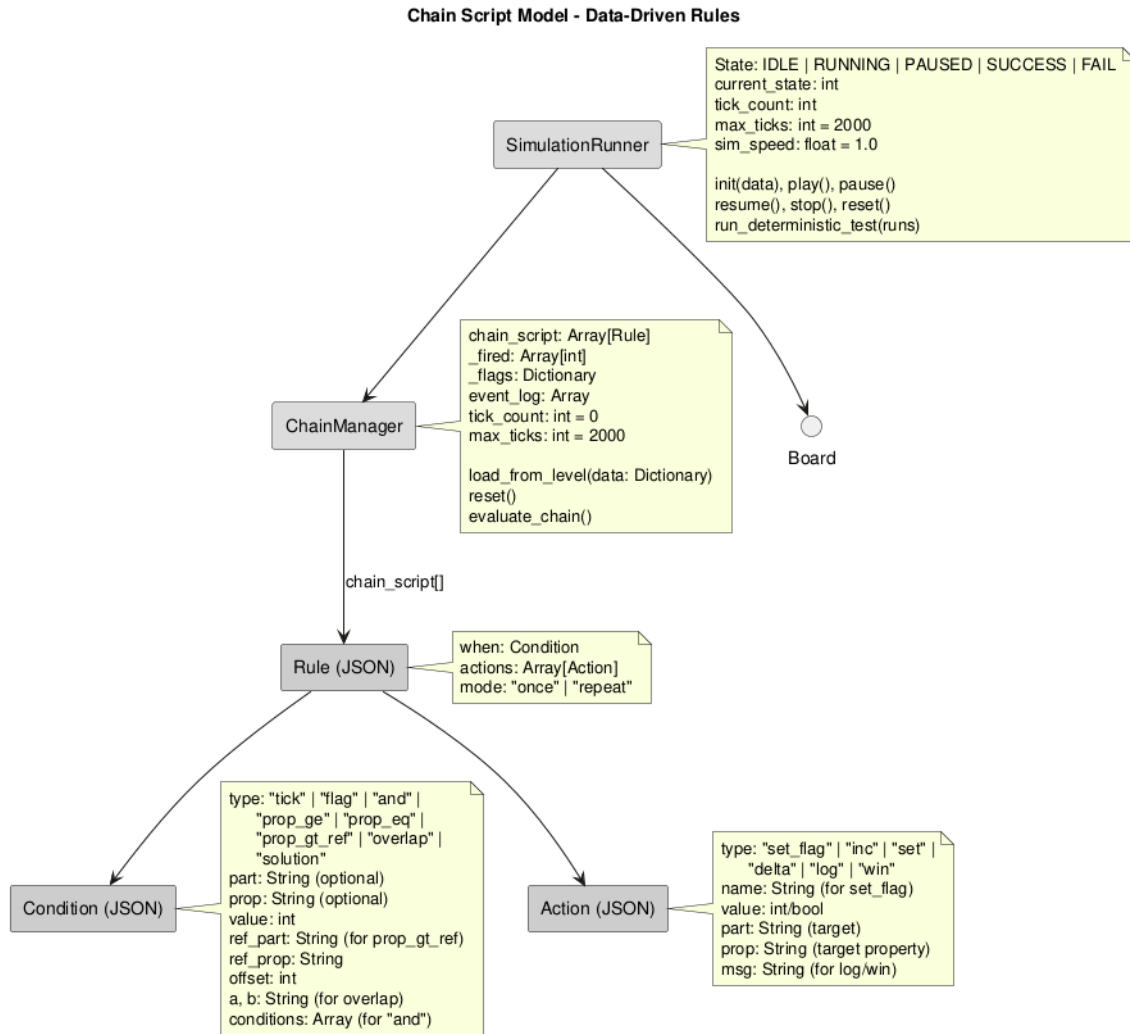


Figure 7: Chain_Script_Model

5. Diagramas de Clase

4.1 Class Diagram – Simulation Architecture

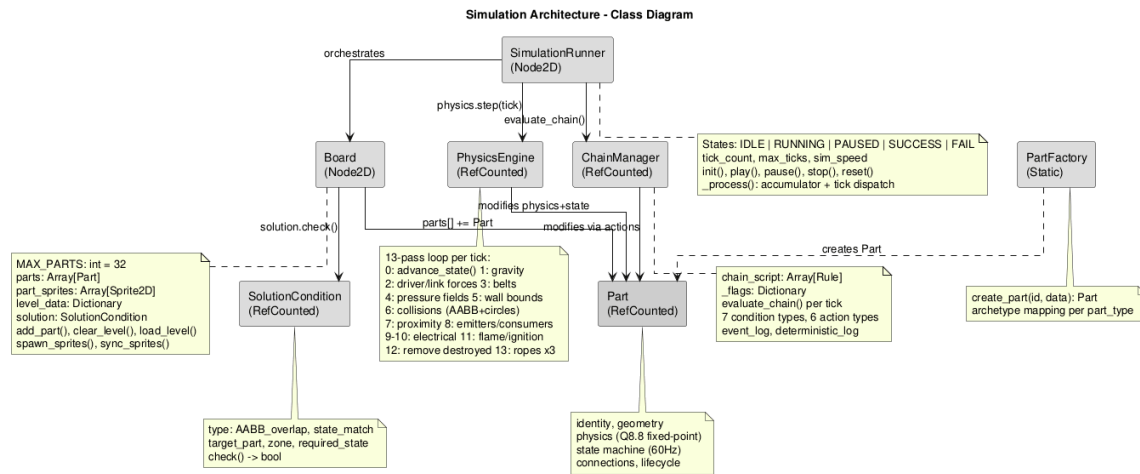


Figure 8: Simulation_Class_Diagram

4.2 Class Diagram – Editor Architecture

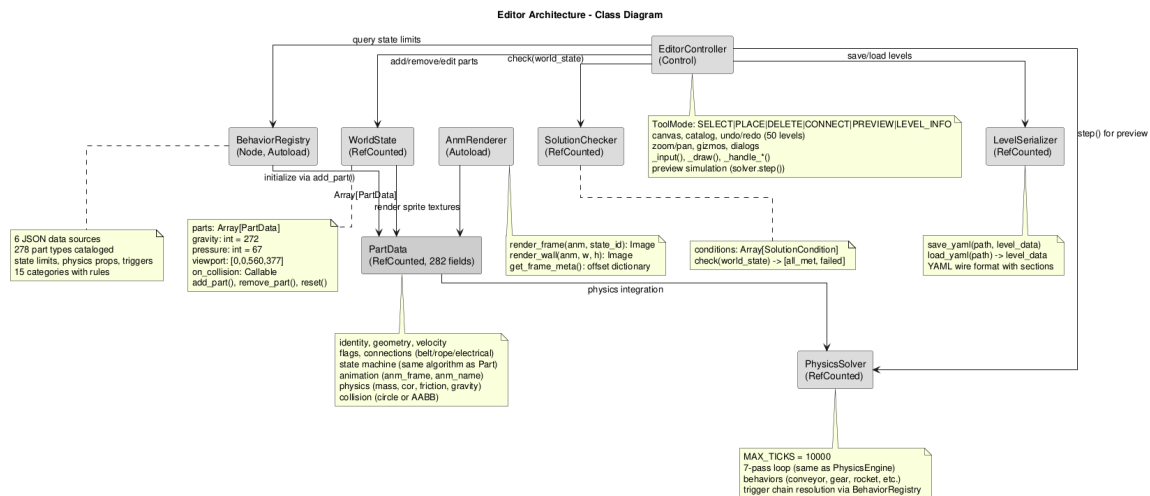


Figure 9: Editor_Class_Diagram

6. Diagramas de Sequência

5.1 Sequence – Simulation Loop (Pacemaker + Chain + Physics)

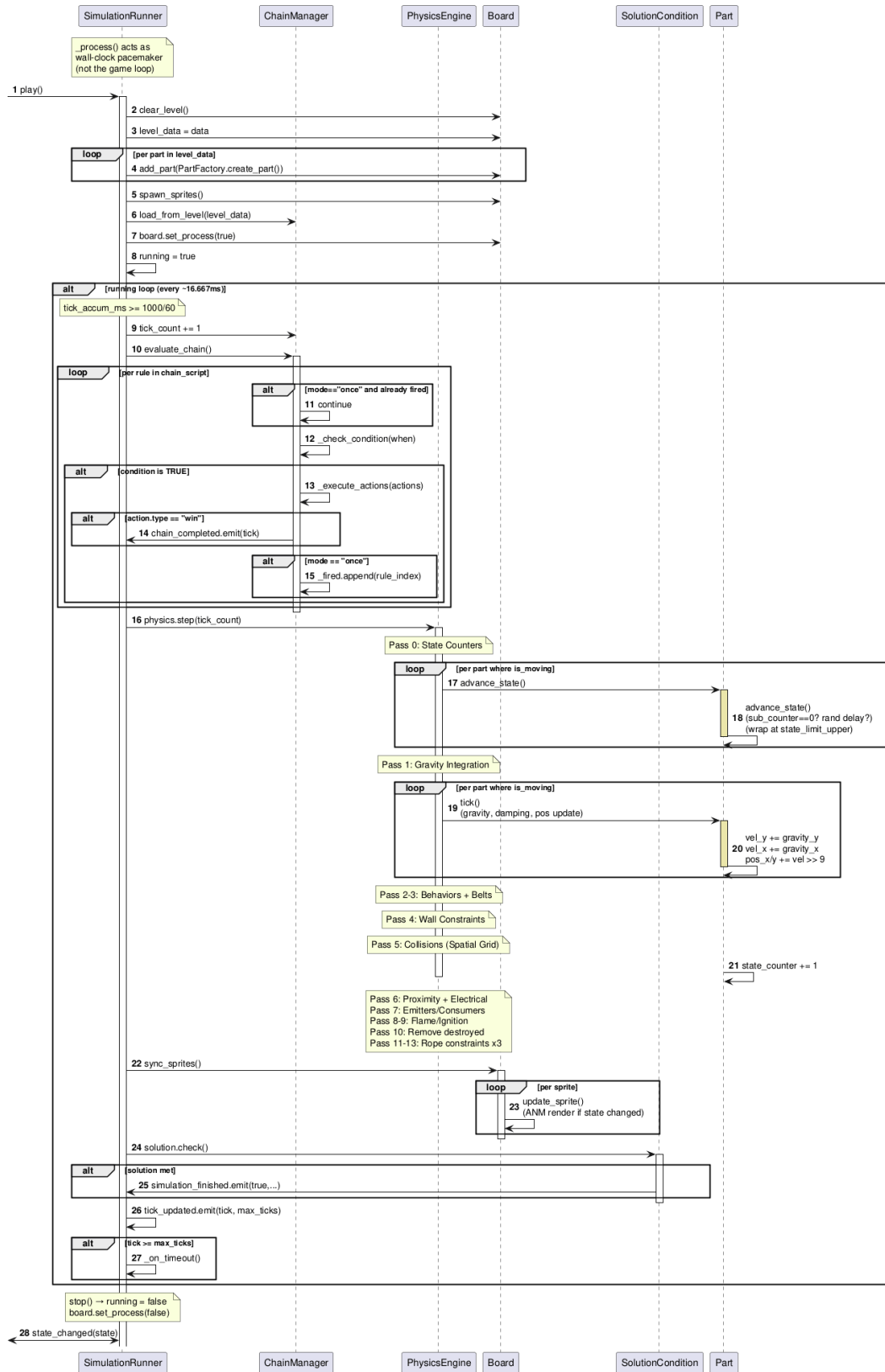


Figure 10: SimulationLoopSequence

5.2 Sequence – Editor Tool Interactions

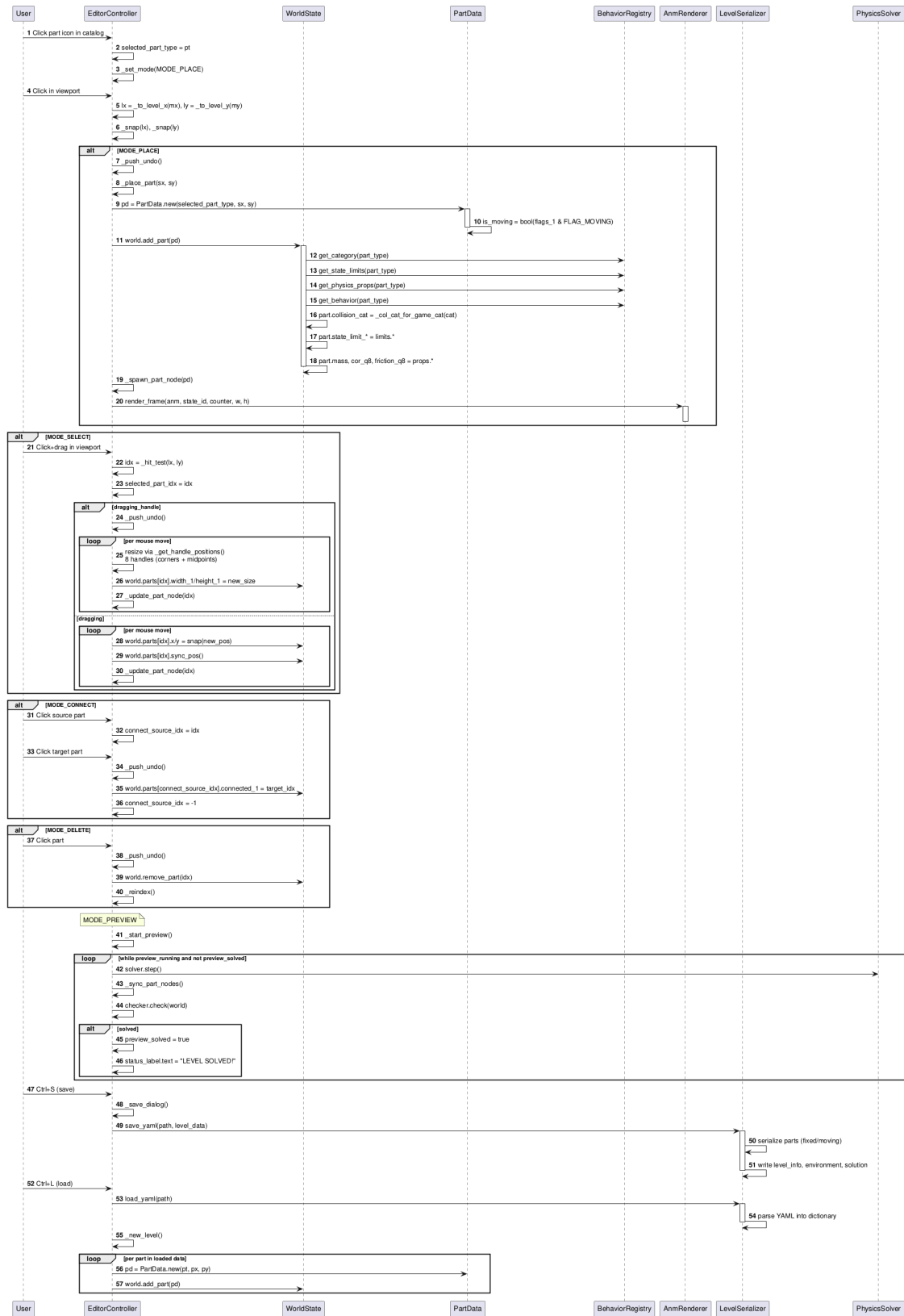


Figure 11: Editor_Tool_Sequence

5.3 Sequence – Chain Script Evaluation

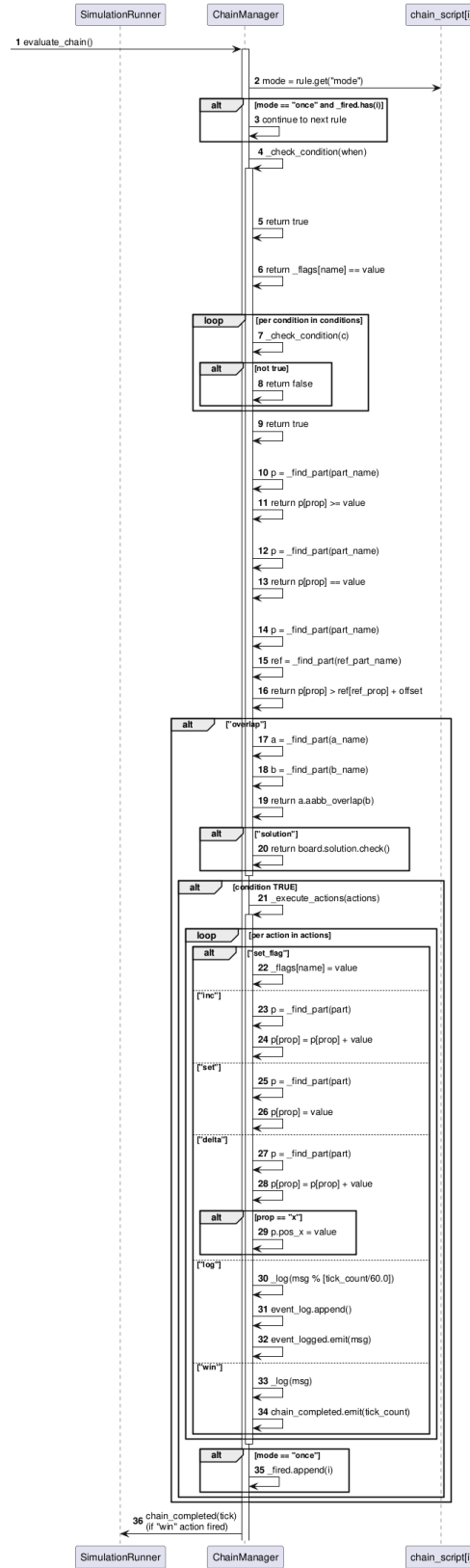


Figure 12: Chain_Evaluation_Sequence

6.1 Part State Machine (Per-Instance, Deterministic)

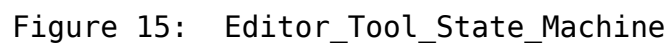
6.1 Part State Machine (Per-Instance, Deterministic)



6.2 SimulationRunner State Machine



6.3 Editor Tool Mode State Machine



8. Wireframes

7.1 Level Editor Wireframe

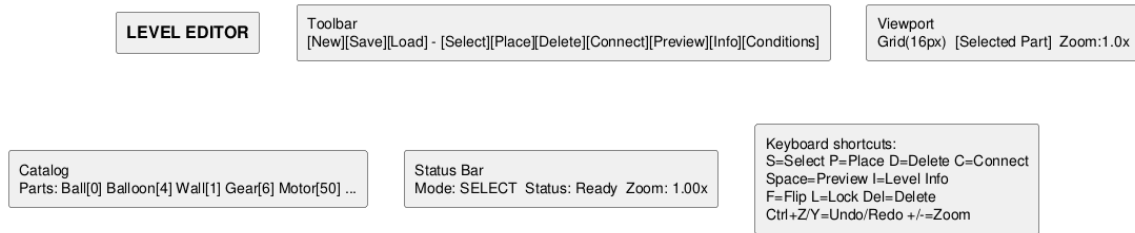


Figure 16: Editor_Wireframe

7.2 Simulation Overlay Wireframe

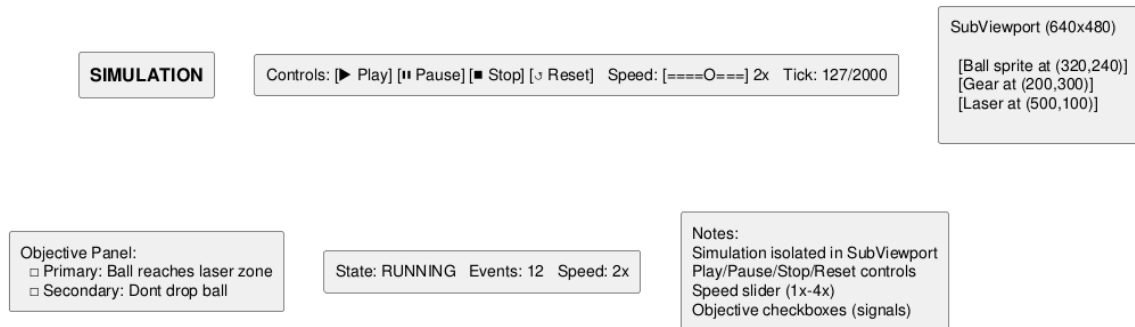


Figure 17: Simulation_Overlay_Wireframe

7.3 Part Gizmos Wireframe

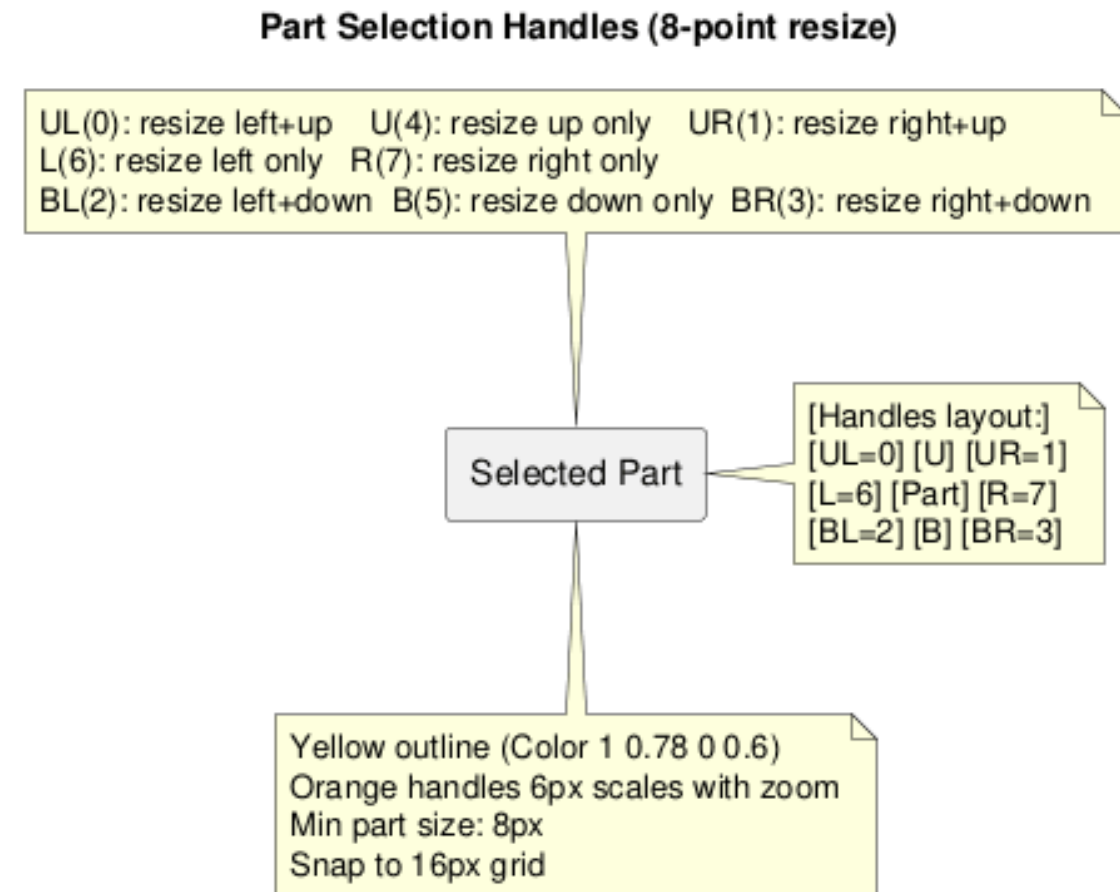


Figure 18: Part_Gizmos_Wireframe

9. Fluxo de Dados

8.1 Data Flow – Editor → Simulation Bridge

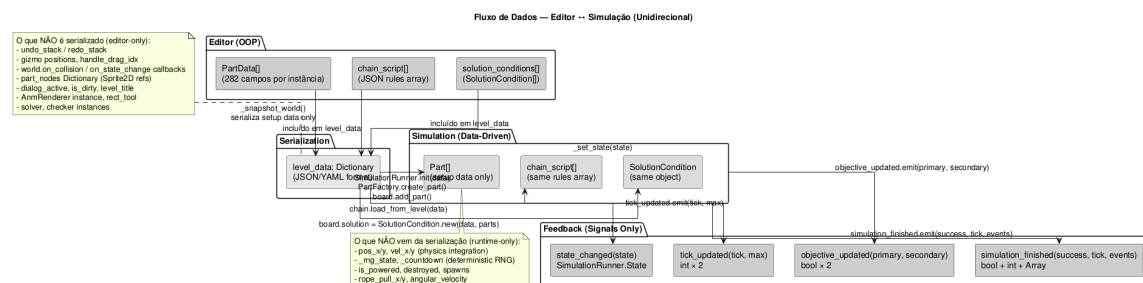


Figure 19: Data_Flow_Bridge

8.2 Physics 7-Pass Data Flow

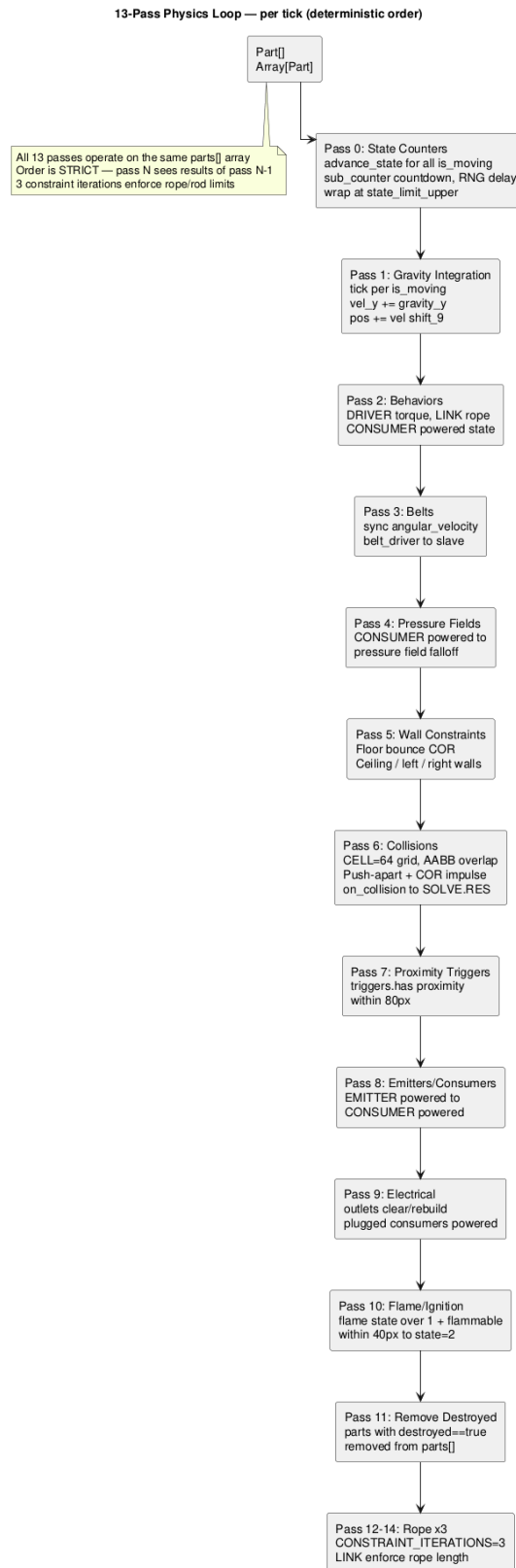


Figure 20: Physics_7Pass_Data_Flow

10. Especificação de APIs

9.1 Part State Machine API

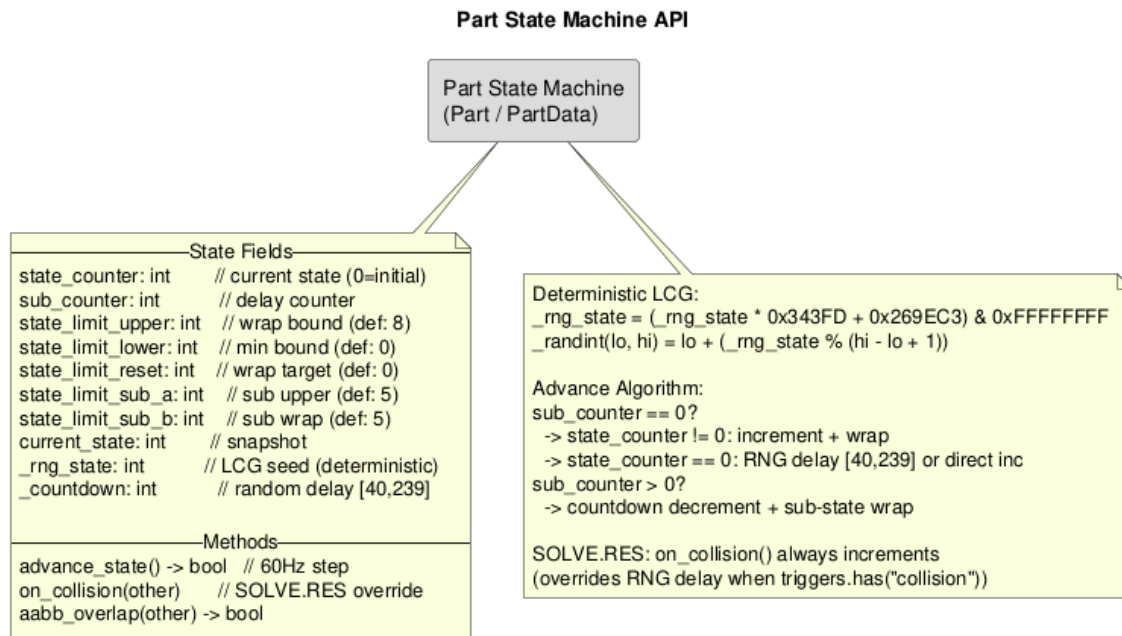
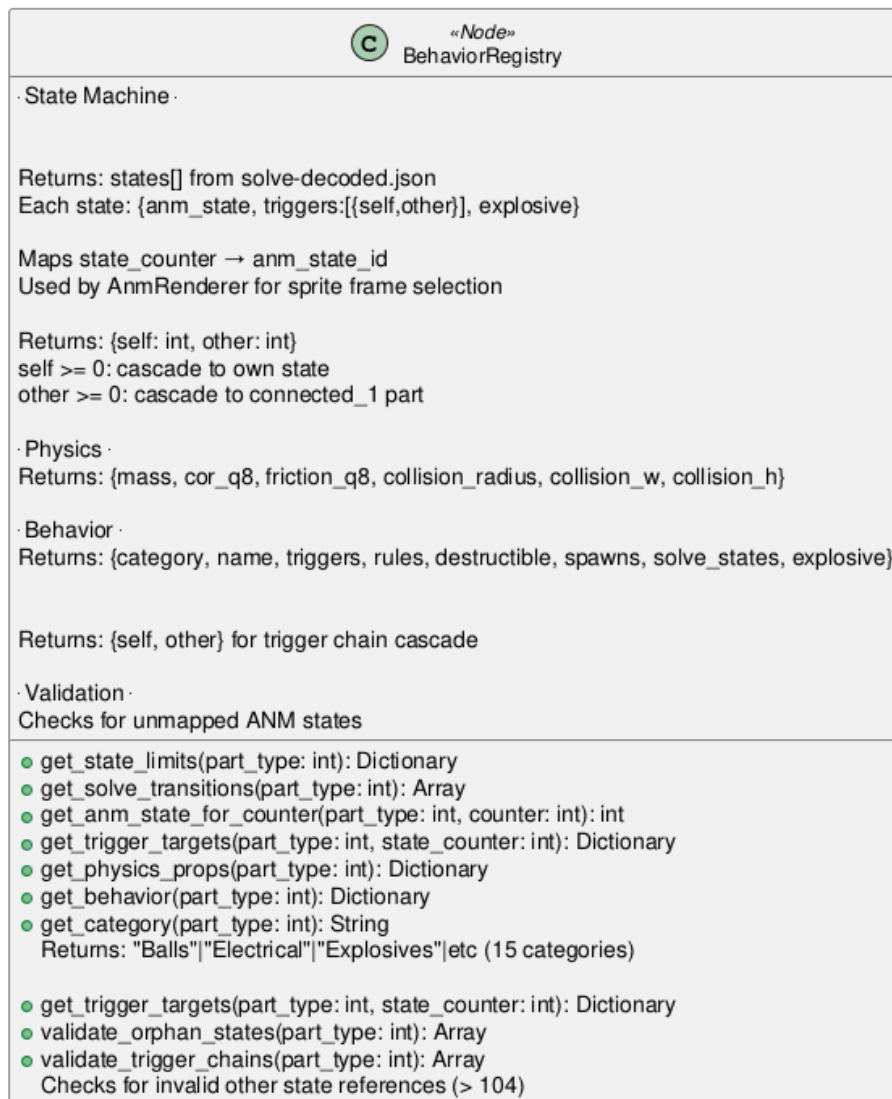


Figure 21: Part_State_Machine_API

9.2 BehaviorRegistry – Key API Surface

BehaviorRegistry — Public API (Autoload)



Data sources (loaded in _ready):

- res://data/solve-decoded.json → state machine definitions
- res://data/behaviors.json → state limits, sub-limits
- res://data/categories.json → category metadata
- res://data/part-properties.json → mass, cor, friction, dimensions
- res://data/part-assets.json → category mapping, sounds
- res://data/state-machine-report.json → orphan ANM states

15 categories:

Balls, Walls/Floors, Inclines, Special Mechanics, Balloons/Airships, Rotating Power Sources, Ropes/Belts/Pulleys, Containers, Characters, Springs/Pneumatics, Explosives/Projectiles, Electrical, Light/Flame Sources, Lasers/Optics, Scenery

Figure 22: BehaviorRegistry_API

11. Análise de Decisões de Implementação

Esta seção documenta as **decisões técnicas de implementação** – escolhas de engenharia que foram feitas durante o desenvolvimento e que explicam por que o código está estruturado da forma que está.

11.1 Por Que Q8.8 Fixed-Point?

Problema: Floating point (float/double) tem comportamento não determinístico em diferentes plataformas e configurações de CPU. Arredondamentos podem variar ligeiramente.

Solução: Q8.8 fixed-point arithmetic – 16 bits integer onde 8 são parte inteira e 8 são parte fracionária.

```
# Exemplo: velocidade em Q8.8
vel_y += gravity_y # gravity_y é Q8.8
pos_y += vel_y >> 8 # shift right 8 = divide por 256
```

Por que 8 bits para a parte fracionária? - Proporciona precisão suficiente para posições e velocidades - 16-bit total cabe em register do CPU sem overflow - Multiplicação de dois Q8.8 ainda cabe em 32-bit

Trade-offs: - **Range limitado:** Max value = 32767.999 ($2^{15} - 1$)
- **Precisão limitada:** $1/256 = 0.0039$ (suficiente para física) -
Benefício: Reproducibilidade entre plataformas

11.2 Por Que 13 Passes em vez de 1 Loop?

Problema: Se todas as física fossem aplicadas em um único loop, a ordem de aplicação seria aleatória (ou dependente da ordem no array).

Solução: 13 passes em ordem estrita e determinística:

Pass	Nome	O que faz
0	State Counters	advance_state() em todos os parts
1	Gravity	Integração gravitacional
2	Behaviors	Drivers (torque), Links (rope pull), Consumers
3	Belts	Sincronização de velocidade angular
4	Pressure Fields	Campos de pressão de consumers
5	Wall Constraints	Bounce no chão/parede/teto
6	Collisions	AABB + circle, impulse resolution
7	Proximity Triggers	Triggers de proximidade
8	Emitters/Consumers	Electrical propagation
9	Electrical	Outlets e consumers
10	Flame/Ignition	Fontes de chama
11	Remove Destroyed	Limpa parts destruídos
12-14	Rope Constraints	3 iterações para ropes/links

Por que a ordem importa? - Gravity deve vir antes de collisions (parts precisam estar em posições corretas) - Collisions deve vir antes de proximity triggers (overlap precisa ser detectado) - Rope constraints deve vir por último (corrige posições após tudo mais) - 3 iterações garantem convergência (cada iteração aproxima mais da solução ideal)

11.3 Por Que Spatial Grid com CELL_SIZE=64?

Problema: Detectar collisions entre N parts é $O(n^2)$ – para 32 parts são 496 checks por frame, 60fps = ~30,000 checks/segundo.

Solução: Spatial grid divide o viewport em células de 64x64 pixels. Cada part é inserido na célula correspondente à sua posição.

```
var grid: Dictionary # key = "cx,cy", value = Array of part indices
```

```
func _insert_in_grid(part, cx, cy):
    var key = "%d,%d" % [cx, cy]
    if not grid.has(key):
        grid[key] = []
    grid[key].append(part.id)
```

Por que CELL_SIZE=64? - Viewport típico é ~640x480 - 64px por célula = ~10x10 grid = 100 células - Cada part checka apenas 9 células vizinhas (3x3 neighborhood) - Reduz de $O(n^2)$ para aproximadamente $O(n * k)$ onde k = parts por célula

Trade-offs: - Parts maiores que 64px podem spanhar múltiplas células - CELL_SIZE=64 é um trade-off entre granularidade e número de células - Para viewport maior, precisaria de células maiores

11.4 Por Que 3 Iterações de Rope Constraints?

Problema: Rope constraints (ARCH_LINK parts) são resolvidores de distância – precisam manter dois parts a uma distância específica.

Solução: Iterative constraint solving com 3 iterações:

```
for iteration in range(3): # CONSTRAINT_ITERATIONS = 3
    for part in parts:
        if part.archetype == ARCH_LINK:
            _enforce_rope_constraint(part, target)
```

Por que 3? - 1 iteração: aproximação grosseira - 2 iterações: melhor, mas ainda não convergido - 3 iterações: suficiente para a maioria dos casos - Mais de 3: custo de performance sem benefício perceptível

Convergência: Constraints são “soft” – cada iteração move os parts um pouco em direção à posição correta. Após 3 iterações, o erro típico é menor que 1px, que é aceitável visualmente.

11.5 Por Que LCG RNG com Seed Fixa?

Contexto: Deterministic RNG é necessário para reprodutibilidade.

Solução: Linear Congruential Generator com seed fixa:

```
_rng_state = (_rng_state * 0x343FD + 0x269EC3) & 0xFFFFFFFF
_randint(lo, hi) = lo + (_rng_state % (hi - lo + 1))
```

Parâmetros escolhidos: - 0x343FD: Multiplicador (escolhido para boa distribuição estatística) - 0x269EC3: Incremento (número primo) - & 0xFFFFFFFF: Mantém em 32-bit unsigned

Por que não Godot's RandomNumberGenerator? - Godot RNG usa plataforma-dependant implementation - Seed fixo não garante determinismo entre plataformas diferentes - LCG é simple, fast, e completamente determinístico

Seed inicial: `_rng_state` começa em 1. Isso significa que qualquer simulação com o mesmo `level_data` produz exatamente os mesmos resultados.

12. Fluxo de Gameplay Completo

Esta seção descreve o **ciclo de vida completo** de uma fase – desde a criação no editor até a resolução no puzzle editor.

12.1 Criação de Fase (Level Editor)

O creator usa o Level Editor para construir uma fase do zero:

1. Nova fase ou carregar existente - `Ctrl+N`: limpa `world_state`, cria fase vazia - `Ctrl+L`: carrega YAML, popula `world_state` com `PartData[]`

2. Selecionar  (tool modes) - **S (SELECT)**: default – clique para selecionar, drag para mover, handles para resize - **P (PLACE)**: seleciona `part_type` do catálogo, clique no viewport para colocar - **D (DELETE)**: clique para remover part, undo disponível - **C (CONNECT)**: primeiro clique = source, segundo clique = target (`connected_1` = target) - **Space (PREVIEW)**: inline simulation running – `solver.step()` por `_process()` - **I (LEVEL_INFO)**: dialog para metadata (title, goal, `bg_color`, pressure, gravity, `music_track`)

3. Adicionar peças - Click no catálogo (`part_type`) - Click no viewport (snapped para 16px grid) - `PartData` criado com `FLAG_MOVING` set - Sprite renderizado via `AnmRenderer`

4. Conectar peças - **SELECT**: seleciona part, mostra 8 handles de resize - **CONNECT**: click source → click target (`connected_1` = `target_idx`) - Visual: linha mostra connection preview

5. Configurar state machine (Condition Editor) - Double-click part: abre `ConditionEditor` dialog - Define `chain_script` rules para esta part - Regras são armazenadas no `level_data`, não no `PartData`

6. Definir solução (Solution Editor) - Dialog para definir objective zones - primary/secondary conditions - Estado requerido (`state_counter` >= X)

7. Testar (PREVIEW mode) - `Space` toggles preview - `Solver.step()` executa física inline - State counters avançam, sprites sync - `SolutionChecker` monitora condições

8. Salvar (Ctrl+S) - YAML salvo com todos os parts, connections, chain_script, solution - is_dirty = false

12.2 Resolução de Puzzle (Puzzle Editor)

0 player recebe uma fase com um inventário limitado de peças:

1. Carregar fase - Carrega same YAML que o creator salvou - Mas: only mostra parts que estão no inventário da fase - Part count é limitado ao que o creator definiu

2. Diferença do Level Editor - Não pode adicionar peças além do inventário - Não pode editar state machines - Não pode alterar connections - Foco: posicionar peças do inventário para satisfazer solution conditions

3. Posicionar peças - Arrasta peças do inventário para o viewport - Snap para 16px grid - Pode rotacionar (R key), flip (F key)

4. Testar (Play button) - Simulação inicia em SubViewport - Physics executa deterministicamente - Player observa se solution conditions são atingidas

5. Iterar - Se falhou: reset, reposicionar, tentar novamente - Se succeeded: level completo, next level unlocked

12.3 Modo de Simulação (SubViewport)

A simulação é isolada em um SubViewport e controlada por UI separada:

1. Inicialização (play())

```
sim_runner.init(level_data) # deserialize
board.clear_level()
board.add_part(PartFactory.create_part(...)) # para cada part
board.spawn_sprites()
chain.load_from_level(level_data)
board.set_process(true)
```

2. Loop de Tick (wall-clock pacemaker)

```
while tick_accum_ms >= 16.667:
    tick_accum_ms -= 16.667
    tick_count += 1
    chain.evaluate_chain() # avalia regras do chain_script
    physics.step(tick_count) # 13 passes de física
    board.sync_sprites() # atualiza sprites se state mudou
```

3. Controles - **Play**: inicia/retoma simulação - **Pause**: pausa pace-maker, física congelada - **Stop**: reseta para IDLE, limpa level - **Reset**: mesmo que stop, mas também reseta chain_script

4. Sinais de feedback

```
signal state_changed(state)      # RUNNING, PAUSED, SUCCESS, FAIL
signal tick_updated(tick, max)   # UI atualiza tick counter
signal objective_updated(primary, secondary) # checkboxes
signal simulation_finished(success, tick, events) # end state
```

5. Timeout - max_ticks default = 2000 - Se tick_count >= max_ticks: FAIL state - Timeout significa que a solução não foi encontrada dentro do tempo esperado

13. Formato de Serialização

Esta seção descreve o **formato de dados** usado para salvar e carregar fases – o bridge entre editor e simulação.

13.1 Por Que YAML?

Decisão: YAML em vez de JSON ou binary.

Vantagens do YAML: - **Legibilidade:** Humans podem ler e editar diretamente - **Version control:** Diff legível no git, fácil resolver conflitos - **Extensibilidade:** Adicionar novos campos sem quebrar parsers antigos - **Tooling:** Libraries disponíveis em todas as linguagens

Desvantagens: - **Performance:** Parsing YAML é mais lento que JSON - **Espaço:** YAML é mais verboso que JSON

Decisão tomada: Legibilidade > performance. Fases são salvas/carregadas infrequently (only ao play e ao save), então o custo de parsing é insignificante.

13.2 Estrutura do level_data

```
level_info:
  title: "Ball in the Basket"
  goal: "Get the ball into the basket"
  bg_color: [200, 220, 240]
  pressure: 67
  gravity: 272
  music_track: "level_01.ogg"
```

```
parts:
  fixed:
```

```

- part_type: 1
  name: "floor"
  x: 0
  y: 300
  width_1: 640
  height_1: 32
  flags_1: 0
  connected_1: -1
  state_limit_upper: 8
  ...

moving:
- part_type: 0
  name: "ball"
  x: 100
  y: 100
  width_1: 32
  height_1: 32
  flags_1: 4096 # FLAG_MOVING
  connected_1: -1
  state_limit_upper: 8
  ...

chain_script:
- when:
  type: "overlap"
  a: "ball"
  b: "basket"
  actions:
  - type: "win"
    msg: "Level Complete!"
  mode: "once"

solution:
  solution_type: "zone_overlap"
  target_part_name: "ball"
  zone:
    x: 500
    y: 280
    w: 80
    h: 40
  required_state: 0

inventory:
- part_type: 0
  count: 3
- part_type: 6
  count: 2

```

13.3 O Que É Serializado vs O Que Não É

Serializado (setup data – vai para simulação):

PartData: part_type, name, x, y, width_1/2, height_1/2,
flags_1/2/3, connected_1/2, state_counter,
state_limit_*, current_state, mass, cor_q8,
friction_q8, collision_radius, facing

NÃO Serializado (editor-only – não vai para simulação):

Editor state:

- undo_stack, redo_stack (50 níveis)
- gizmo positions, handle_drag_idx
- world.on_collision, on_state_change (Callable)
- part_nodes (Sprite2D refs no editor)
- dialog_active, is_dirty, level_title
- rect_tool (Selection rectangle tool)
- solver instance, checker instance

Runtime-only (gerado na simulação):

- pos_x, pos_y, vel_x, vel_y (physics integration)
- _rng_state, _countdown (deterministic RNG state)
- is_powered, destroyed, spawns
- rope_pull_x, rope_pull_y

13.4 LevelSerializer – Save/Load

class LevelSerializer extends RefCounted:

```
func save_yaml(path: String, level: Dictionary):
    # Serialize parts: fixed[] e moving[]
    # Serialize chain_script[]
    # Serialize solution{}
    # Serialize level_info{}
    # Serialize inventory[]

func load_yaml(path: String) -> Dictionary:
    # Parse YAML
    # Validate required fields
    # Return level_data dictionary
    # Não cria PartData --- isso é feito pelo caller
```

Formato de saída: - Seção “fixed”: parts com FLAG_MOVING não setado
- Seção “moving”: parts com FLAG_MOVING setado - Cada part list é um array de dicionários com campos em hex16 (exceto coordenadas e nomes)

Hex16 encoding: Valores numéricos são salvos como hex strings para legibilidade e compactação. Ex: state_limit_upper: 8 VS state_limit_upper: "0x8".

14. Resumo de Constraints

14.1 Constraints Verificadas

Constraint	Implementação
Determinismo 50/50	<code>SimulationRunner.run_deterministic_test(50)</code> → PASS
60Hz fixed tick	<code>tick_accum_ms >= 1000/60</code> no <code>_process()</code> <code>pacemaker</code>
Q8.8 fixed-point	<code>vel_y >> 9, cor_q8 >> 8, gravity table</code>
7-pass physics	<code>PhysicsEngine.step()</code> – 13 passes (0-12)
Dual representation	<code>PartData</code> (282 campos) ↔ <code>Part</code> (60 campos)
OOP editor	<code>EditorController</code> com 6 tool modes, undo/redo
Data-driven simulation	<code>chain_script</code> array de regras JSON
State machines first-class	<code>advance_state()</code> em <code>Part</code> e <code>PartData</code>
Signal-based feedback	<code>state_changed,</code> <code>tick_updated, simulation_finished</code>
SubViewport isolation	<code>SimulationOverlay.tscn</code> (<code>SubViewport</code> + <code>controls</code>)
No runtime state sharing	Serializa apenas setup data na bridge

Documento de Arquitetura – Grandma’s Garden Versão expandida com seções narrativas de decisões de design Última atualização: Introdução, Decisões Arquiteturais, Análise de Implementação, Gameplay Flow, Serialização